

Лабораторная работа №3

Шифрование гаммированием

Кадрова Ангелина Александровна

Содержание

1 Введение	4
1.1 Цели и задачи	4
1.2 Теоретическое введение	4
2 Выполнение лабораторной работы	5
3 Вывод	8
Список литературы	9

Список иллюстраций

2.1 Результаты работы шифра	7
---------------------------------------	---

1 Введение

1.1 Цели и задачи

Цель работы

Основная цель работы — изучить и реализовать шифрование гаммированием.

Задание

С помощью языка программирования Julia реализовать алгоритм шифрования гаммированием конечной гаммой.

1.2 Теоретическое введение

Julia — высокоуровневый свободный язык программирования с динамической типизацией, созданный для математических вычислений[1]. Эффективен также и для написания программ общего назначения. Синтаксис языка схож с синтаксисом других математических языков, однако имеет некоторые существенные отличия. Для выполнения заданий была использована официальная документация Julia[2].

2 Выполнение лабораторной работы

Шифрование гаммированием — симметричный метод шифрования, при котором к открытым данным(тексту) применяется операция наложения с последовательностью чисел, называемой гаммой.

```
alphabet = collect("абвгдежзийклмнопрстуфхцщъыьэюя ")
```

```
MOD = 33
```

```
function char_to_num(c)
    return findfirst(==(c), alphabet)
end
```

```
function num_to_char(n)
    if n == 0
        n = MOD
    end
    return alphabet[n]
end
```

```
function encrypt(text, gamma)
    text_chars = collect(lowercase(text))
    gamma_chars = collect(lowercase(gamma))

    encrypted = ""
```

```

    for i in 1:length(text_chars)
        p = char_to_num(text_chars[i])
        k = char_to_num(gamma_chars[(i - 1) % length(gamma_chars) + 1])
        c = (p + k) % MOD
        encrypted *= num_to_char(c)
    end

    return encrypted
end

function decrypt(cryptogram, gamma)
    crypto_chars = collect(lowercase(cryptogram))
    gamma_chars = collect(lowercase(gamma))

    decrypted = ""

    for i in 1:length(crypto_chars)
        c = char_to_num(crypto_chars[i])
        k = char_to_num(gamma_chars[(i - 1) % length(gamma_chars) + 1])
        p = mod(c - k, MOD)
        decrypted *= num_to_char(p)
    end

    return decrypted
end

function main()
    text = "приказ"

```

```

gamma = "гамма"

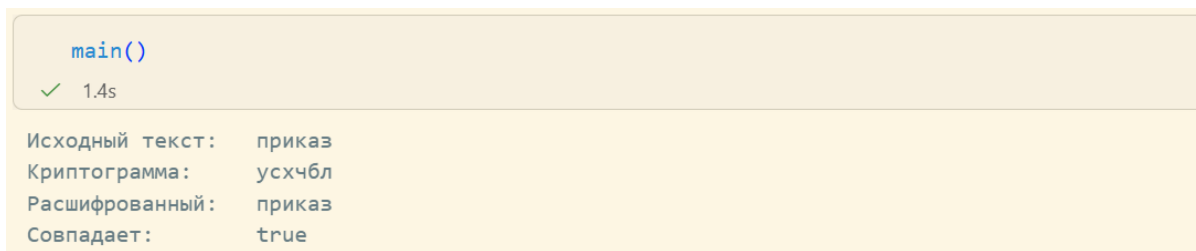
cryptogram = encrypt(text, gamma)
decrypted  = decrypt(cryptogram, gamma)

println("Исходный текст:   ", text)
println("Криптограмма:    ", cryptogram)
println("Расшифрованный:   ", decrypted)
println("Совпадает:       ", decrypted == text)
end

main()

```

В результате получим следующее(рис. 2.1):



The screenshot shows a code editor with the function `main()` and its execution results. The code defines a gamma key and performs encryption and decryption. The output shows the original text 'приказ' being encrypted to 'усхчбл' and then decrypted back to 'приказ', with a confirmation that they match.

```

main()
✓ 1.4s
Исходный текст:   приказ
Криптограмма:    усхчбл
Расшифрованный:   приказ
Совпадает:       true

```

Рис. 2.1: Результаты работы шифра

3 Вывод

С помощью языка программирования Julia был реализован алгоритм шифрования гаммированием конечной гаммой.

Список литературы

1. JuliaLang [Электронный ресурс]. 2026 JuliaLang.org contributors. URL: <https://julialang.org/> (дата обращения: 10.10.2026).
2. Julia 1.13 Documentation [Электронный ресурс]. 2026 JuliaLang.org contributors. URL: <https://docs.julialang.org/en/v1/> (дата обращения: 10.10.2026).